

README

VisionEngine

Computer-vision + LLM-vision toolkit for UI analysis and navigation-graph construction. Decoupled from any consuming project per CONST-051(B); incorporated by HelixCode + HelixQA as an equal-codebase submodule per CONST-051(A).

Overview

VisionEngine provides four cooperating layers:

- **Analyzer** (pkg/analyzer) — Analyzer interface, VideoProcessor interface, value types (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame), StubAnalyzer reference implementation with CONST-046 i18n seam.
- **NavigationGraph** (pkg/graph) — directed graph for tracking app screen transitions with BFS pathfinding, three export back-ends (DOT, JSON, Mermaid), plus stress / automation / integration / security test suites.
- **LLM Vision providers** (pkg/llmvision) — VisionProvider interface and adapters for OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama; FallbackChain composer.
- **Configuration** (pkg/config) — env-var loader + validator with every user-facing error string routed through pkg/i18n.Translator.

Build-tag gating: pkg/opencv ships stubs in the default build path and real GoCV bindings under `-tags vision`. The rest of the module is buildable + testable WITHOUT OpenCV on any Go 1.25.3+ host.

Quick start

```
# Run tests (no OpenCV required)
go test -race -count=1 ./...
```

```

# Build
go build ./...

# With OpenCV support
go build -tags vision ./...

# Exercise the public surface end-to-end with the round-297 runner
go run ./challenges/runner/

# Paired-mutation Challenge (anti-bluff proof – see CONST-035)
./challenges/scripts/visionengine_describe_challenge.sh
    # exit 0 (normal)
./challenges/scripts/visionengine_describe_challenge.sh --mutate
    # exit 99 (mutation detected)

```

Packages

Package	Purpose
pkg/analyzer	Analyzer interface, value types, StubAnalyzer reference impl, CONST-046 Translator seam
pkg/graph	NavigationGraph with BFS PathTo, ExportDOT / ExportJSON / ExportMermaid, full test matrix
pkg/llmvision	VisionProvider + 9 cloud / local adapters (OpenAI, Anthropic, Gemini, Qwen, Kimi, StepGUI, Astica, Ollama) + FallbackChain
pkg/opencv	OpenCV stubs (default) + real GoCV bindings behind -tags vision
pkg/config	Env-driven configuration loader + i18n-routed validation
pkg/i18n	Minimal dependency-free Translator interface + NoopTranslator standalone default
pkg/remote	VisionPool SSH wiring for distributed vision workers (round-40 anti-bluff close-out at commit 8dbf6fd)

NavigationGraph (most-imported by HelixQA)

```

g := graph.NewNavigationGraph()
g.AddScreen(analyzer.ScreenIdentity{ID: "home", Name: "Home"})
g.AddScreen(analyzer.ScreenIdentity{ID: "settings", Name:
    "Settings"})
g.AddTransition("home", "settings", analyzer.Action{Type:
    "click", Target: "gear"})
g.SetCurrent("home")

```

```
path, _ := g.PathTo("settings")
fmt.Println(graph.ExportMermaid(g))
```

Anti-bluff guarantees

VisionEngine is governed by Article XI §11.9 + CONST-035: every PASS in this module's test + Challenge suites MUST carry positive runtime evidence captured during execution.

1. **No mocks beyond unit tests** (CONST-050(A)). pkg/analyzer/stub.go is the canonical reference implementation, not a placeholder. The round-297 challenge runner (challenges/runner/main.go) constructs real StubAnalyzer + NavigationGraph instances and asserts on real return values; no test double substitutes for the public API.
2. **Negative-path assertions are mandatory.** AnalyzeScreen(nil) MUST return analyzer.ErrEmptyScreenshot; runner check 2 fails the whole Challenge if the sentinel is missing or wrong.
3. **5-locale bilingual evidence** per CONST-046. Every captured runner line carries [en | sr | ja | de | es] labels so artefacts cannot be English-only smoke. The locale strings live in the runner (a consumer of the module), NOT inside pkg/, so the module itself stays language-agnostic.
4. **Paired-mutation Challenge** (visionengine_describe_challenge.sh --mutate). The script plants a deliberate regression in a scratch copy of pkg/graph/graph.go (zeroing AddTransition) and asserts the runner FAILs. If the runner spuriously passes on mutated source, the Challenge is itself a bluff and exits 2 to flag it. Normal mode exits 0, mutate mode exits 99 — these are the canonical round-220 paired-mutation codes.
5. **CONST-046 Translator seam.** Every user-facing string (AnalyzeScreen Title/Description, config validation errors, FallbackChain error wrappers, provider validation errors) resolves through pkg/i18n.Translator. NoopTranslator (the standalone-default) returns the msgID verbatim so the call site falls back to its bundled English fixture — keeping the module project-not-aware per CONST-051(B).
6. **CONST-051(B) decoupling.** The module imports nothing from HelixCode / HelixQA / any consuming project. Verify: !
grep -r 'helix_code\|helixqa\|HelixCode\|HelixQA' pkg/ go.mod.
7. **Symbol→test ledger** at docs/test-coverage.md maps every exported symbol in pkg/{analyzer,graph,llmvision,opencv,config,i18n,remote} to the test (or Challenge) that exercises it. Gaps (e.g. VideoProcessor interface has no impl yet) are listed honestly rather than pretended-covered.

Reproducing the evidence

```
cd dependencies/HelixDevelopment/VisionEngine
go test -race -count=1 ./...
go build ./...
./challenges/scripts/visionengine_describe_challenge.sh
./challenges/scripts/visionengine_describe_challenge.sh --mutate
```

Captured exit codes + last-line evidence MUST be pasted in the PR or commit body that closes any work touching this module, per Article XI §11.9. See docs/test-coverage.md for the round-297 reference capture.

Governance cascade

This submodule inherits the universal Constitution from constitution/ at the meta-repo root and tightens it where the submodule's scope is narrower. Project-specific clauses MUST never appear in this submodule's source per CONST-051(B); they live in the consuming project (HelixCode, HelixQA).

Relevant constitutional anchors:

- **CONST-035** — Zero-bluff mandate (Article XI §11.9 forensic anchor)
- **CONST-046** — No hardcoded user-facing content (Translator seam)
- **CONST-050(A)** — No fakes beyond unit tests
- **CONST-051(B)** — Decoupling / reusability mandate
- **CONST-053** — .gitignore + no versioned build artefacts
- **CONST-054** — Submodule-dependency manifest (see helix-deps.yaml when present at root)
- **CONST-060** — Fetch-before-edit (first git action every session)

License

Apache License 2.0. See LICENSE.